

§ 6.5 图遍历算法概述

图算法是个庞大的家族，其中大部分成员的主体框架，都可归结于图的遍历。与5.4节中树的遍历类似，图的遍历也需要访问所有顶点一次且仅一次；此外，图遍历同时还需要访问所有的边一次且仅一次——尽管对树而言这显而易见——并对边做分类，以便后续的处理。

实际上，无论采用何种策略和算法，图的遍历都可理解为，将非线性结构转化为半线性结构的过程。经遍历而确定的边类型中，最重要的一类即所谓的树边，它们与所有顶点共同构成了原图的一棵支撑树（森林），称作遍历树（**traversal tree**）。以遍历树为背景，其余各种类型的边，也能提供关于原图的重要信息，比如其中所含的环路等。

图中顶点之间可能存在多条通路，故为避免对顶点的重复访问，在遍历的过程中，通常还要动态地设置各顶点不同的状态，并随着遍历的进程不断地转换状态，直至最后的“访问完毕”。图的遍历更加强调对处于特定状态顶点的甄别与查找，故也称作图搜索（**graph search**）。

与树遍历一样，作为图算法基石的图搜索，本身也必须能够高效地实现。幸运的是，正如我们马上就会看到的，诸如深度优先、广度优先、最佳优先等基本而典型的图搜索，都可以在线性时间内完成。准确地，若顶点数和边数分别为 n 和 e ，则这些算法自身仅需 $O(n + e)$ 时间。既然图搜索需要访问所有的顶点和边，故这已经是我们所能期望的最优的结果。

§ 6.6 广度优先搜索

6.6.1 策略

各种图搜索之间的区别，体现为边分类结果的不同，以及所得遍历树（森林）的结构差异。其决定因素在于，搜索过程中的每一步迭代，将依照何种策略来选取下一接受访问的顶点。

通常，都是选取某个已访问到的顶点的邻居。同一顶点所有邻居之间的优先级，在多数遍历中不必讲究。因此，实质的差异应体现在，当有多个顶点已被访问到，应该优先从谁的邻居中选取下一顶点。比如，广度优先搜索（**breadth-first search, BFS**）采用的策略，可概括为：

越早被访问到的顶点，其邻居越优先被选用

于是，始自图中顶点 s 的BFS搜索，将首先访问顶点 s ；再依次访问 s 所有尚未访问到的邻居；再按后者被访问的先后次序，逐个访问它们的邻居；...；如此不断。在所有已访问到的顶点中，仍有邻居尚未访问者，构成所谓的波峰集（**frontier**）。于是，BFS搜索过程也可等效地理解为：

反复从波峰集中找到最早被访问到顶点 v ，若其邻居均已访问到，则将其逐出波峰集；否则，随意选出一个尚未访问到的邻居，并将其加入到波峰集中

不难发现，若将上述BFS策略应用于树结构，则效果等同于层次遍历（5.4.5节）——波峰集内顶点的深度始终相差不超过一，且波峰集总是优先在更浅的层次沿广度方向拓展。实际上，树层次遍历的这些特性，在一定程度上也适用于图的BFS搜索（习题[6-7]）。

由于每一步迭代都有一个顶点被访问，故至多迭代 $O(n)$ 步。另一方面，因为不会遗漏每个刚被访问顶点的任何邻居，故对于无向图必能覆盖 s 所属的连通分量（**connected component**），对于有向图必能覆盖以 s 为起点的可达分量（**reachable component**）。倘若还有来自其它连通分量或可达分量的顶点，则不妨从该顶点出发，重复上述过程。

6.6.2 实现

图的广度优先搜索算法，可实现如代码6.3所示。

```

1  template <typename Tv, typename Te> //广度优先搜索BFS算法 ( 全图 )
2  void Graph<Tv, Te>::bfs(int s) { //assert: 0 <= s < n
3      reset(); int clock = 0; int v = s; //初始化
4      do //逐一检查所有顶点
5          if (UNDISCOVERED == status(v)) //一旦遇到尚未发现的顶点
6              BFS(v, clock); //即从该顶点出发启动一次BFS
7      while (s != (v = (++v % n))); //按序号检查, 故不漏不重
8  }
9
10 template <typename Tv, typename Te> //广度优先搜索BFS算法 ( 单个连通域 )
11 void Graph<Tv, Te>::BFS(int v, int& clock) { //assert: 0 <= v < n
12     Queue<int> Q; //引入辅助队列
13     status(v) = DISCOVERED; Q.enqueue(v); //初始化起点
14     while (!Q.empty()) { //在Q变空之前, 不断
15         int v = Q.dequeue(); dTime(v) = ++clock; //取出队首顶点v
16         for (int u = firstNbr(v); -1 < u; u = nextNbr(v, u)) //枚举v的所有邻居u
17             if (UNDISCOVERED == status(u)) { //若u尚未被发现, 则
18                 status(u) = DISCOVERED; Q.enqueue(u); //发现该顶点
19                 status(v, u) = TREE; parent(u) = v; //引入树边拓展支撑树
20             } else { //若u已被发现, 或者甚至已访问完毕, 则
21                 status(v, u) = CROSS; //将(v, u)归类于跨边
22             }
23         status(v) = VISITED; //至此, 当前顶点访问完毕
24     }
25 }
```

代码6.3 BFS算法

算法的实质功能，由子算法BFS()完成。对该函数的反复调用，即可遍历所有连通或可达域。仿照树的层次遍历，这里也借助队列Q，来保存已被发现，但尚未访问完毕的顶点。因此，任何顶点在进入该队列的同时，都被随即标记为DISCOVERED（已发现）状态。

BFS()的每一步迭代，都先从Q中取出当前的首顶点v；再逐一核对其各邻居u的状态并做相应处理；最后将顶点v置为VISITED（访问完毕）状态，即可进入下一步迭代。

若顶点u尚处于UNDISCOVERED（未发现）状态，则令其转为DISCOVERED状态，并随即加入队列Q。实际上，每次发现一个这样的顶点u，都意味着遍历树可从v到u拓展一条边。于是，将边(v, u)标记为树边（tree edge），并按照遍历树中的承袭关系，将v记作u的父节点。

若顶点u已处于DISCOVERED状态（无向图），或者甚至处于VISITED状态（有向图），则意味着边(v, u)不属于遍历树，于是将该边归类为跨边（cross edge）（习题[6-11]）。

BFS()遍历结束后，所有访问过的顶点通过parent[]指针依次联接，从整体上给出了原图某一连通或可达域的一棵遍历树，称作广度优先搜索树，或简称BFS树（BFS tree）。

6.6.3 实例

图6.7给出了一个含8个顶点和11条边的有向图，起始于顶点S的BFS搜索过程。请留意观察辅助队列（下方）的演变，顶点状态的变化，边的分类与结果，以及BFS树的生长过程。

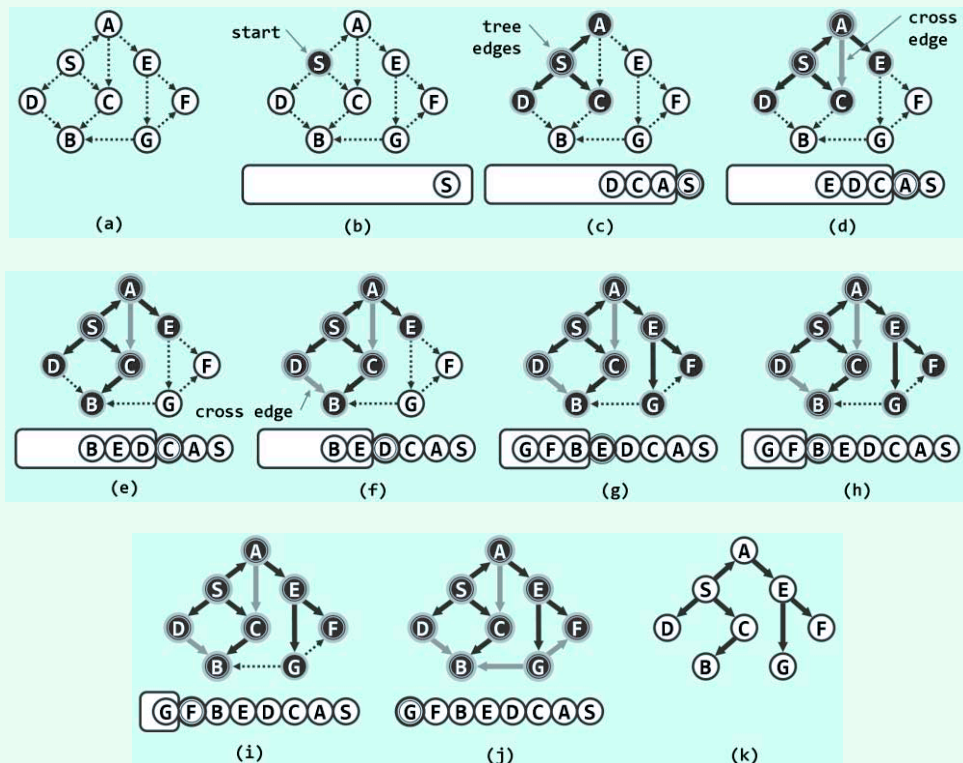


图6.7 广度优先搜索示例

不难看出， $\text{BFS}(s)$ 将覆盖起始顶点 s 所属的连通分量或可达分量，但无法抵达此外的顶点。而上层主函数 $\text{bfs}()$ 的作用，正在于处理多个连通分量或可达分量并存的情况。具体地，在逐个检查顶点的过程中，只要发现某一点尚未被发现，则意味着其所属的连通分量或可达分量尚未触及，故可从该顶点出发再次启动 $\text{BFS}()$ ，以遍历其所属的连通分量或可达分量。如此，各次 $\text{BFS}()$ 调用所得的BFS树构成一个森林，称作BFS森林（BFS forest）。

6.6.4 复杂度

除作为输入的图本身外，BFS搜索所使用的空间，主要消耗在用于维护顶点访问次序的辅助队列、用于记录顶点和边状态的标识位向量，累计 $O(n) + O(n) + O(e) = O(n + e)$ 。

时间方面，首先需花费 $O(n + e)$ 时间复位所有顶点和边的状态。不计对子函数 $\text{BFS}()$ 的调用， $\text{bfs}()$ 本身对所有顶点的枚举共需 $O(n)$ 时间。而在对 $\text{BFS}()$ 的所有调用中，每个顶点、每条边均只耗费 $O(1)$ 时间，累计 $O(n + e)$ 。综合起来，BFS搜索总体仅需 $O(n + e)$ 时间。

6.6.5 应用

基于BFS搜索，可有效地解决连通域分解（习题[6-6]）、最短路径（习题[6-8]）等问题。

§ 6.7 深度优先搜索

6.7.1 策略

深度优先搜索（Depth-First Search, DFS）选取下一顶点的策略，可概括为：

优先选取最后一个被访问到的顶点的邻居

于是，以顶点 s 为基点的DFS搜索，将首先访问顶点 s ；再从 s 所有尚未访问到的邻居中任取其一，并以之为基点，递归地执行DFS搜索。故各顶点被访问到的次序，类似于树的先序遍历（5.4.2节）；而各顶点被访问完毕的次序，则类似于树的后序遍历（5.4.4节）。

6.7.2 实现

深度优先遍历算法可实现如代码6.4所示。

```

1 template <typename Tv, typename Te> //深度优先搜索DFS算法（全图）
2 void Graph<Tv, Te>::dfs(int s) { //assert: 0 <= s < n
3     reset(); int clock = 0; int v = s; //初始化
4     do //逐一检查所有顶点
5         if (UNDISCOVERED == status(v)) //一旦遇到尚未发现的顶点
6             DFS(v, clock); //即从该顶点出发启动一次DFS
7     while (s != (v = (++v % n))); //按序号检查，故不漏不重
8 }
9
10 template <typename Tv, typename Te> //深度优先搜索DFS算法（单个连通域）
11 void Graph<Tv, Te>::DFS(int v, int& clock) { //assert: 0 <= v < n
12     dTime(v) = ++clock; status(v) = DISCOVERED; //发现当前顶点v
13     for (int u = firstNbr(v); -1 < u; u = nextNbr(v, u)) //枚举v的所有邻居u
14         switch (status(u)) { //并视其状态分别处理
15             case UNDISCOVERED: //u尚未发现，意味着支撑树可在此拓展
16                 status(v, u) = TREE; parent(u) = v; DFS(u, clock); break;
17             case DISCOVERED: //u已被发现但尚未访问完毕，应属被后代指向的祖先
18                 status(v, u) = BACKWARD; break;
19             default: //u已访问完毕（VISITED，有向图），则视承袭关系分为前向边或跨边
20                 status(v, u) = (dTime(v) < dTime(u)) ? FORWARD : CROSS; break;
21         }
22     status(v) = VISITED; fTime(v) = ++clock; //至此，当前顶点v方告访问完毕
23 }
```

代码6.4 DFS算法

算法的实质功能，由子算法DFS()递归地完成。每一递归实例中，都先将当前节点 v 标记为DISCOVERED（已发现）状态，再逐一核对其各邻居 u 的状态并做相应处理。待其所有邻居均已处理完毕之后，将顶点 v 置为VISITED（访问完毕）状态，便可回溯。

若顶点 u 尚处于UNDISCOVERED（未发现）状态，则将边 (v, u) 归类为树边（tree edge），